



```
def main(src: cown[Account], dst: cown[Account]) {
```

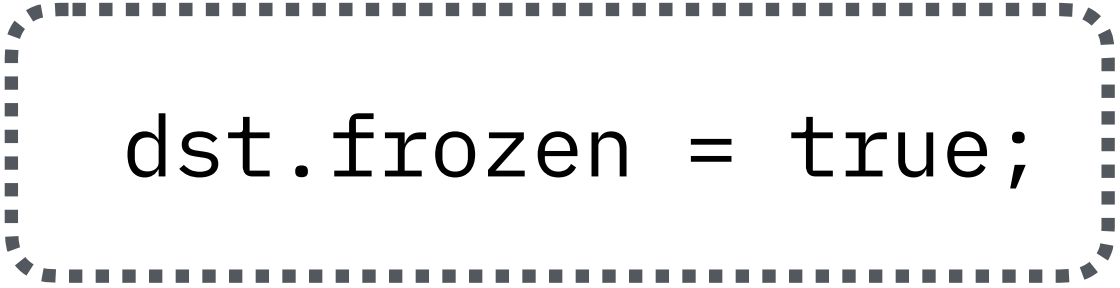


```
  when(dst) { B dst.frozen = true; }
```

```
}
```

```
def main(src: cown[Account], dst: cown[Account]) {  
    when(src) {  src.balance += 10; }  
    when(dst) {  dst.frozen = true; }  
}
```





```
dst.frozen = true;
```



```
src.balance += 10;
```







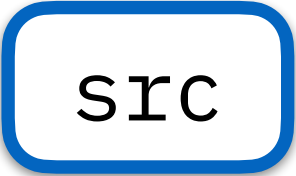
A



B

balance: 0

frozen: false



src

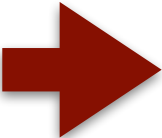
balance: 0

frozen: false

dst

BoC: Owns and Behaviours

Spawning is synch but executing is async




```
def main(src: Account, dst: Account) {
```

src:balance += 10; }

when(sirc) }

dst: frozen == true; }







A



B







A

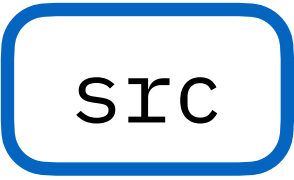


B



balance: 0

frozen: false



src

balance: 0

frozen: false

dst

Spawning is synchronous but executing is asynchronous

What are available but with a cost to the owner's state

